1. Normalization in Deep Learning

Normalization is carried out in the input layer to normalize the raw data. However, for deep neural nets, the activation values of the middle layer nodes can get very large, causing the same problem as our example. This necessitates some type of normalization in the hidden layers as well [1], as shown in Figure 1. Let x be that tensor consisting of a batch of N images. Each of these images has C feature maps or channels with height H and weight W. Therefore, $x \in \mathbb{R}^{N \times C \times H \times W}$ is a four-dimensional tensor.

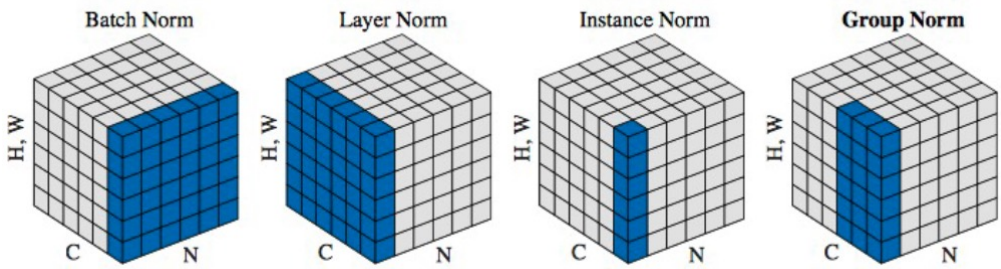

Figure 1. An overview of the normalization methods.

1.1. Batch Normalization.

BN operates on one channel (one feature map) over all the training samples in the mini-batch. BN normalizes its activation values using the mean (μ_c) and variance (σ_c^2) of the mini-batch and introduces two learnable parameters, γ and β. The network can learn these parameters using a Gradient Descent algorithm.

$$\mu_c = \frac{1}{NHW} \sum_{n=1}^{N} \sum_{w=1}^{W} \sum_{h=1}^{H} x_{ncwh} \tag{1}$$

$$\sigma_c^2 = \frac{1}{NHW} \sum_{n=1}^{N} \sum_{w=1}^{W} \sum_{h=1}^{H} (x_{ncwh} - \mu_c)^2 \tag{2}$$

We add the ϵ in computing $\hat{x}_{ncwh}$ to the denominator to stabilize the training in case the variance of the mini-batch is zero.

$$\hat{x}_{ncwh} = \frac{x_{ncwh} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} \tag{3}$$

Finally,

$$y_{ncwh} = \gamma \hat{x}_{ncwh} + \beta, \tag{4}$$

where γ and β parameters are the scaling and bias operators. This transformation preserves the network's capacity by ensuring that the normalization layer can produce the identity function. Both normalization methods can accelerate training and make the network converge faster. Plus, BN enables larger training rates and improves the generalization of the neural network. BN depends on the batch size in that, to obtain a statistically more accurate mean and variance, the batch size needs to be large. In addition, since BN is dependent on the batch size, it can't be applied at test time the same way as the training time. The reason is that in the test phase, we usually have one example to process. Therefore, the mean and variance can't be computed like training. Instead, during test time, BN utilizes moving average and variance to perform inference.

1.2. **Instance Normalization.** Instance normalization was first coined in the StyleNet paper [2]. Instance normalization tells us that it operates on a single sample. To clarify how IN works, let's consider sample feature maps that constitute an input tensor to the IN layer. In instance normalization, we consider one training sample and feature map and take the mean and variance over its spatial locations (W and H).
To perform instance normalization for a single instance x_{nc}, we need to compute the mean and variance:

$$\mu_{nc} = \frac{1}{HW} \sum_{w=1}^{W} \sum_{h=1}^{H} x_{ncwh} \tag{5}$$

$$\sigma_{nc}^2 = \frac{1}{HW} \sum_{w=1}^{W} \sum_{h=1}^{H} (x_{ncwh} - \mu_{nc})^2. \tag{6}$$

Then, using the obtained mean and variance, each spatial dimension is normalized:

$$\hat{x}_{ncwh} = \frac{x_{ncwh} - \mu_{nc}}{\sqrt{\sigma_{nc}^2 + \epsilon}}, \tag{7}$$

where ϵ is a small value added for more stable training. In the end, using two scaling and bias parameters, we linearly transform the result.
While BN transforms the whole mini-batch of samples, IN does that to a single training sample. A small batch size setting is where IN performs better than BN. IN doesn't depend on the batch size, so its implementation is kept the same for both training and testing.

1.3. **Layer Normalization.** In BN, the statistics are computed across the batch and the spatial dims. In contrast, in Layer Normalization (LN), the statistics (mean and variance) are computed across all channels and spatial dims. Thus, the statistics are independent of the batch. This layer was initially introduced to handle vectors (mostly the RNN outputs).
Nearly nobody spoke about LN until the Transformers paper came out. We can take the mean across the spatial dimension and across all channels, as illustrated below:

$$\mu_n = \frac{1}{CHW} \sum_{c=1}^{C} \sum_{w=1}^{W} \sum_{h=1}^{H} x_{ncwh} \tag{8}$$

$$\sigma_n^2 = \frac{1}{CHW} \sum_{c=1}^{C} \sum_{w=1}^{W} \sum_{h=1}^{H} (x_{ncwh} - \mu_n)^2. \tag{9}$$

Then, using the obtained mean and variance, each spatial dimension is normalized:

$$\hat{x}_{ncwh} = \frac{x_{ncwh} - \mu_n}{\sqrt{\sigma_n^2 + \epsilon}}, \tag{10}$$

where ϵ is a small value added for more stable training. In the end, using two scaling and bias parameters, we linearly transform the result.

1.4. **Group Normalization.** Group normalization (GN) divides the channels into groups and computes the first-order statistics within each group. As a result, GN's computation is independent of batch sizes, and its accuracy is more stable than BN in a wide range of batch sizes.
For $groups = number$ of channels we get instance normalization, while for $groups = 1$ the method is reduced to layer normalization.

$$\mu_i = \frac{1}{m} \sum_{k \in \mathcal{S}_i} x_k, \tag{11}$$

$$\sigma_i^2 = \frac{1}{m}\sum_{k\in\mathcal{S}_i}(x_k - \mu_{i]})^2, \tag{12}$$

where

$$\mathcal{S}_i = \{k|k_N = i_N, \lfloor\frac{k_C}{C/G}\rfloor = \lfloor\frac{i_C}{C/G}\rfloor\}. \tag{13}$$

Note that G is the number of groups, which is a hyper-parameter. C/G is the number of channels per group. Thus, GN computes μ and σ along the (H, W) axes and along a group of C/G channels.

1.5. **Adaptive Instance Normalization.** Normalization and style transfer are closely related. In Instance Normalization, what if γ and β are injected from the feature statistics of another image y? In this way, we will be able to model any arbitrary style by just giving our desired feature image mean as β and variance as γ from style image y.
Adaptive Instance Normalization (AdaIN) receives an input image x (content) and a style input y, and simply aligns the channel-wise mean and variance of x to match those of y. Mathematically:

$$AdaIN(x, y) = \sigma(y)\frac{x - \mu(x)}{\sigma(x)} + \mu(y) \tag{14}$$

References

[1] Nikolas Adaloglou. In-layer normalization techniques for training very deep neural networks. *https://theaisummer.com/*, 2020.

[2] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Improved texture networks: Maximizing quality and diversity in feed-forward stylization and texture synthesis. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 6924–6932, 2017.